- Qt Core
- Qt GUI
- Qt Network
- Qt Qml
- Qt Quick
- Qt Widgets
  - Widgets
  - Window and Dialog Widgets
  - Dialog Windows
  - Styles and Style Aware Widgets
  - Qt Style Sheets
    - The Style Sheet Syntax
    - Qt Widgets Designer Integra
    - Using Style Sheets
    - Style Sheets Reference
    - Style Sheets Examples
  - Layout Management
  - Model/View Programming
  - Keyboard Focus in Widgets
  - Gestures in Widgets and Graphi
  - Graphics View Framework
  - Upgrading form Qt 5
  - Tutorials
  - Examples
  - C++ Classes
- Qt Test
- Modules
- Tools and utilities

Contents

Style Rules
Selector Types
Sub-Controls
Pseudo-States
Conflict Resolution
Cascading
Inheritance
Widgets Inside C++ Namespaces
Setting QObject Properties

Next
Qt Widgets Designer Integration
Previous
Qt Style Sheets

The Style Sheet Syntax

Qt Style Sheet terminology and syntactic rules are almost identical to those of HTML CSS. If you already know CSS, you can probably skim quickly through this section.

Style Rules

Style sheets consist of a sequence of style rules. A *style rule* is made up of a selector and a declaration. The *selector* specifies which widgets are affected by the rule; the *declaration* specifies which properties should be set on the widget. For example:

```
QPushButton { color: red }
```

In the above style rule, `QPushButton` is the selector and `{ color: red }` is the declaration. The rule specifies that **QPushButton** and its subclasses (e.g., `MyPushButton`) should use red as their foreground color.

Qt Style Sheet is generally case insensitive (i.e., `color`, `Color`, `COLOR`, and `cOloR` refer to the same property). The only exceptions are class names, **object names**, and Qt property names, which are case sensitive.

Several selectors can be specified for the same declaration, using commas (`,`) to separate the selectors. For example, the rule

```
QPushButton, QLineEdit, QComboBox { color: red }
```

is equivalent to this sequence of three rules:

```
QPushButton { color: red }
QLineEdit { color: red }
QComboBox { color: red }
```

The declaration part of a style rule is a list of *`property: value`* pairs, enclosed in braces (`{}`) and separated with semicolons. For example:

```
QPushButton { color: red; background-color: white }
```

See the **List of Properties** section below for the list of properties provided by Qt widgets.

Selector Types

All the examples so far used the simplest type of selector, the Type Selector. Qt Style Sheets support all the **selectors defined in CSS2**. The table below summarizes the most useful types of selectors.

Selector	Example	Explanation
Universal Selector	`*`	Matches all widgets.
Type Selector	`QPushButton`	Matches instances of **QPushButton** and of its subclasses.
Property Selector	`QPushButton[flat="false"]`	Matches instances of **QPushButton** that are not **flat**. You may use this selector to test for any Qt **property** that supports **QVariant::toString**() (see the **toString**() function documentation for details). In addition, the special `class` property is supported, for the name of the class. This selector may also be used to test dynamic properties. For more information on customization using dynamic properties, refer to **Customizing Using Dynamic Properties**. Instead of `=`, you can also use `~=` to test whether a Qt property of type **QStringList** contains a given **QString**. **Warning:** If the value of the Qt property changes after the style sheet has been set, it might be necessary to force a style sheet recomputation. One way to achieve this is to unset the style sheet and set it again.
Class Selector	`.QPushButton`	Matches instances of **QPushButton**, but not of its subclasses. This is equivalent to `*[class~="QPushButton"]`.
ID Selector	`QPushButton#okButton`	Matches all **QPushButton** instances whose **object name** is `okButton`.
Descendant Selector	`QDialog QPushButton`	Matches all instances of **QPushButton** that are descendants (children, grandchildren, etc.) of a **QDialog**.
Child Selector	`QDialog > QPushButton`	Matches all instances of **QPushButton** that are direct children of a **QDialog**.

Sub-Controls

For styling complex widgets, it is necessary to access subcontrols of the widget, such as the drop-down button of a **QComboBox** or the up and down arrows of a **QSpinBox**. Selectors may contain *subcontrols* that make it possible to restrict the application of a rule to specific widget subcontrols. For example:

```
QComboBox::drop-down { image: url(dropdown.png) }
```

The above rule styles the drop-down button of all **QComboBox**es. Although the double-colon (`::`) syntax is reminiscent of CSS3 Pseudo-Elements, Qt Sub-Controls differ conceptually from these and have different cascading semantics.

Sub-controls are always positioned with respect to another element - a reference element. This reference element could be the widget or another Sub-control. For example, the **::drop-down** of a **QComboBox** is placed, by default, in the top right corner of the Padding rectangle of the **QComboBox**. The **::drop-down** is placed, by default, in the Center of the Contents rectangle of the **::drop-down** Sub-control. See the **List of Stylable Widgets** below for the Sub-controls to use to style a widget and their default positions.

The origin rectangle to be used can be changed using the **subcontrol-origin** property. For example, if we want to place the drop-down in the margin rectangle of the **QComboBox** instead of the default Padding rectangle, we can specify:

```
QComboBox {
    margin-right: 20px;
}
QComboBox::drop-down {
    subcontrol-origin: margin;
}
```

The alignment of the drop-down within the Margin rectangle is changed using **subcontrol-position** property.

The **width** and **height** properties can be used to control the size of the Sub-control. Note that setting a **image** implicitly sets the size of a Sub-control.

The relative positioning scheme (**position** : relative), allows the position of the Sub-Control to be offset from its initial position. For example, when the **QComboBox**'s drop-down button is pressed, we might like the arrow inside to be offset to give a "pressed" effect. To achieve this, we can specify:

```
QComboBox::down-arrow {
    image: url(down_arrow.png);
}
QComboBox::down-arrow:pressed {
    position: relative;
    top: 1px; left: 1px;
}
```

The absolute positioning scheme (**position** : absolute), allows the position and size of the Sub-control to be changed with respect to the reference element.

Once positioned, they are treated the same as widgets and can be styled using the **box model**.

See the **List of Sub-Controls** below for a list of supported sub-controls, and **Customizing the QPushButton's Menu Indicator Sub-Control** for a realistic example.

> **Note:** With complex widgets such as **QComboBox** and **QScrollBar**, if one property or sub-control is customized, **all** the other properties or sub-controls must be customized as well.

Pseudo-States

Selectors may contain *pseudo-states* that denote that restrict the application of the rule based on the widget's state. Pseudo-states appear at the end of the selector, with a colon (`:`) in between. For example, the following rule applies when the mouse hovers over a **QPushButton**:

```
QPushButton:hover { color: white }
```

Pseudo-states can be negated using the exclamation operator. For example, the following rule applies when the mouse does not hover over a **QRadioButton**:

```
QRadioButton:!hover { color: red }
```

Pseudo-states can be chained, in which case a logical AND is implied. For example, the following rule applies to when the mouse hovers over a checked **QCheckBox**:

```
QCheckBox:hover:checked { color: white }
```

Negated Pseudo-states may appear in Pseudo-state chains. For example, the following rule applies when the mouse hovers over a **QPushButton** that is not pressed:

```
QPushButton:hover:!pressed { color: blue; }
```

If needed, logical OR can be expressed using the comma operator:

```
QCheckBox:hover, QCheckBox:checked { color: white }
```

Pseudo-states can appear in combination with subcontrols. For example:

```
QComboBox::drop-down:hover { image: url(dropdown_bright.png) }
```

See the **List of Pseudo-States** section below for the list of pseudo-states provided by Qt widgets.

Conflict Resolution

Conflicts arise when several style rules specify the same properties with different values. Consider the following style sheet:

```
QPushButton#okButton { color: gray }
QPushButton { color: red }
```

Both rules match **QPushButton** instances called `okButton` and there is a conflict for the `color` property. To resolve this conflict, we must take into account the *specificity* of the selectors. In the above example, `QPushButton#okButton` is considered more specific than `QPushButton`, because it (usually) refers to a single object, not to all instances of a class.

Similarly, selectors with pseudo-states are more specific than ones that do not specify pseudo-states. Thus, the following style sheet specifies that a **QPushButton** should have white text when the mouse is hovering over it, otherwise red text:

```
QPushButton:hover { color: white }
QPushButton { color: red }
```

Here's a tricky one:

```
QPushButton:hover { color: white }
QPushButton:enabled { color: red }
```

Here, both selectors have the same specificity, so if the mouse hovers over the button while it is enabled, the second rule takes precedence. If we want the text to be white in that case, we can reorder the rules like this:

```
QPushButton:enabled { color: red }
QPushButton:hover { color: white }
```

Alternatively, we can make the first rule more specific:

```
QPushButton:hover:enabled { color: white }
QPushButton:enabled { color: red }
```

A similar issue arises in conjunction with Type Selectors. Consider the following example:

```
QPushButton { color: red }
QAbstractButton { color: gray }
```

Both rules apply to **QPushButton** instances (since **QPushButton** inherits **QAbstractButton**) and there is a conflict for the **color** property. Because **QPushButton** inherits **QAbstractButton**, it might be tempting to assume that `QPushButton` is more specific than `QAbstractButton`. However, for style sheet computations, all Type Selectors have the same specificity, and the rule that appears last takes precedence. In other words, **color** is set to `gray` for all **QAbstractButton**s, including **QPushButton**s. If we really want **QPushButton**s to have red text, we can always reorder the rules.

For determining the specificity of a rule, Qt Style Sheets follow the **CSS2 Specification**:

> *A selector's specificity is calculated as follows:*
>
> - *count the number of ID attributes in the selector (= a)*
> - *count the number of other attributes and pseudo-classes in the selector (= b)*
> - *count the number of element names in the selector (= c)*
> - *ignore pseudo-elements [i.e., **subcontrols**].*
>
> *Concatenating the three numbers a-b-c (in a number system with a large base) gives the specificity.*
>
> *Some examples:*
>
> ```
> *             {}  /* a=0 b=0 c=0 -> specificity =   0 */
> LI            {}  /* a=0 b=0 c=1 -> specificity =   1 */
> UL LI         {}  /* a=0 b=0 c=2 -> specificity =   2 */
> UL OL+LI      {}  /* a=0 b=0 c=3 -> specificity =   3 */
> H1 + *[REL=up]{}  /* a=0 b=1 c=1 -> specificity =  11 */
> UL OL LI.red  {}  /* a=0 b=1 c=3 -> specificity =  13 */
> LI.red.level  {}  /* a=0 b=2 c=1 -> specificity =  21 */
> #x34y         {}  /* a=1 b=0 c=0 -> specificity = 100 */
> ```

Cascading

Style sheets can be set on the **QApplication**, on parent widgets, and on child widgets. An arbitrary widget's effective style sheet is obtained by merging the style sheets set on the widget's ancestors (parent, grandparent, etc.), as well as any style sheet set on the **QApplication**.

When conflicts arise, the widget's own style sheet is always preferred to any inherited style sheet, irrespective of the specificity of the conflicting rules. Likewise, the parent widget's style sheet is preferred to the grandparent's, etc.

One consequence of this is that setting a style rule on a widget automatically gives it precedence over other rules specified in the ancestor widgets' style sheets or the **QApplication** style sheet. Consider the following example. First, we set a style sheet on the **QApplication**:

```
qApp->setStyleSheet("QPushButton { color: white }");
```

Then we set a style sheet on a **QPushButton** object:

```
myPushButton->setStyleSheet("* { color: blue }");
```

The style sheet on the **QPushButton** forces the **QPushButton** (and any child widget) to have blue text, in spite of the more specific rule set provided by the application-wide style sheet.

The result would have been the same if we had written

```
myPushButton->setStyleSheet("color: blue");
```

except that if the **QPushButton** had children (which is unlikely), the style sheet would have no impact on them.

Style sheet cascading is a complex topic. Refer to the **CSS2 Specification** for the gory details. Be aware that Qt currently doesn't implement `!important`.

Inheritance

In classic CSS, when font and color of an item is not explicitly set, it gets automatically inherited from the parent. By default, when using Qt Style Sheets, a widget does **not** automatically inherit its font and color setting from its parent widget.

For example, consider a **QPushButton** inside a **QGroupBox**:

```
qApp->setStyleSheet("QGroupBox { color: red; } ");
```

The **QPushButton** does not have an explicit color set. Hence, instead of inheriting color of its parent **QGroupBox**, it has the system color. If we want to set the color on a **QGroupBox** and its children, we can write:

```
qApp->setStyleSheet("QGroupBox, QGroupBox * { color: red; }");
```

In contrast, setting a font and palette using **QWidget::setFont**() and **QWidget::setPalette**() propagates to child widgets.

If you would prefer that the font and palette propagate to child widgets, you can set the **Qt::AA_UseStyleSheetPropagationInWidgetStyles** flag, like this:

Usage:

```
QCoreApplication::setAttribute(Qt::AA_UseStyleSheetPropagationInWidgetStyles, true);
```

When the widget-style font and palette propagation is enabled, font and palette changes made through Qt Style Sheets will behave as if the user had manually called the corresponding **QWidget::setPalette**() and **QWidget::setFont**() methods on all of the QWidgets targeted by the style sheet.

- Changes made by a style sheet are propagated. They are pushed to all widgets matching the style sheet once, at the time the change is made.
- Changes made by calling **QWidget::setPalette**() or **QWidget::setFont**() are inherited. They are inherited by all existing and future children, where the respective brush or font hasn't been explicitly set.

Widgets Inside C++ Namespaces

The Type Selector can be used to style widgets of a particular type. For example,

```
class MyPushButton : public QPushButton {
    // ...
}

// ...
qApp->setStyleSheet("MyPushButton { background: yellow; }");
```

Qt Style Sheet uses QObject::className() of the widget to determine when to apply the Type Selector. When custom widgets are inside namespaces, the QObject::className() returns <namespace>::<classname>. This conflicts with the syntax for **Sub-Controls**. To overcome this problem, when using the Type Selector for widgets inside namespaces, we must replace the `::` with `--`. For example,

```
namespace ns {
    class MyPushButton : public QPushButton {
        // ...
    }
}

// ...
qApp->setStyleSheet("ns--MyPushButton { background: yellow; }");
```

Setting QObject Properties

From 4.3 and above, any designable **Q_PROPERTY** can be set using the qproperty-<property name> syntax.

For example,

```
MyLabel { qproperty-pixmap: url(pixmap.png); }
MyGroupBox { qproperty-titleColor: rgb(100, 200, 100); }
QPushButton { qproperty-iconSize: 20px 20px; }
```

If the property references an enum declared with **Q_ENUM**, you should reference its constants by name, not their numeric value.

> **Note:** Use the qproperty syntax with care, as it modifies the widget that is being painted. Also, the qproperty syntax is evaluated only once, which is when the widget is polished by the style. This means that any attempt to use them in pseudo-states such as **QPushButton**:hover, will not work.

< Qt Style Sheets　　Qt Widgets Designer Integration >

© 2025 The Qt Company Ltd. Documentation contributions included herein are the copyrights of their respective owners. The documentation provided herein is licensed under the terms of the GNU Free Documentation License version 1.3 as published by the Free Software Foundation. Qt and respective logos are trademarks of The Qt Company Ltd. in Finland and/or other countries worldwide. All other trademarks are property of their respective owners.

Qt Group

Contact Us

Qt Group
Our Story
Brand
News
Careers
Investors
Qt Products
Quality Assurance Products

Licensing
License Agreement
Open Source
Plans and pricing
Download
FAQ

Learn Qt
For Learners
For Students and Teachers
Qt Documentation
Qt Forum

Support & Services
Professional Services
Customer Success
Support Services
Partners
Qt World

Feedback

© 2025 The Qt Company
Qt Group includes The Qt Company Oy and its global subsidiaries and affiliates.